Lab 9: LLM vs Agent

Single-Agent vs. Orchestrated LLM Workflows for Software Engineering

RICARDO ANDRES CALVO MENDEZ, Purdue University, USA

Multi-agent orchestration frameworks are reshaping how LLMs are applied to software engineering, but coordinating specialized roles introduces real token overhead. This lab asks a direct question: *when does that overhead pay off?*

Rather than building a capable agent system, students will implement a minimal agentic workflow over a real codebase and rigorously compare it against a single-agent baseline. The primary metric is the so-called **CostToSuccess**: total tokens spent until the first acceptable solution is produced (see Section 2.2.2). Students will empirically determine when orchestration helps versus when it is pure overhead.

This lab has three parts: a pre-lab, an in-class component, and a take-home component. All students are expected to complete the pre-lab and in-class component. The take-home component counts toward the "complete 3 take-home labs" requirement.

1 Lab Purpose and Context

As the era of generative AI progresses, we are moving beyond using LLMs in the old-timey "ChatGPT window" manner. Software engineering increasingly leverages bounded, autonomous systems to resolve issues, write patches, and analyze codebases. However, adding specialized roles (*e.g.* Planner, Implementer, Reviewer) also adds coordination cost.

Software engineering is increasingly relying on autonomous bounded LLM systems to resolve issues, write patches, and analyze codebases. Adding specialized roles, such as Planner, Implementer, or Reviewer, can reduce wasted search and catch errors early. But coordinating those roles is costly. Whether that cost is worth paying depends on the complexity of the task.

This lab compares two bounded LLM-based engineering workflows over the same tasks: a single-agent baseline and a simple orchestrated workflow. We test a single hypothesis with two parts:

(1) A plain LLM should be more cost-effective on simple tasks, because it avoids orchestration overhead.
(2) A bounded multi-agent workflow should become more cost-effective on harder tasks, because decomposition and review reduce wasted search and failed attempts.

The primary metric is **CostToSuccess**: total tokens spent until the first acceptable solution. In the in-class component, you will run both conditions against three pre-solved GitHub issues of increasing complexity. In the take-home component, you will extend this to harder tasks, probing for the crossover point at which orchestration becomes cost-effective.

1.1 Learning Objectives

After completing this lab, you should be able to:

- Implement a minimal agentic engineering workflow over a real codebase.
- Compare a single-agent baseline against a bounded orchestrated workflow.
- Measure token cost as a primary evaluation criterion in addition to perceived output quality.
- Identify the limits of LLM evaluation and the value of imposed control structure.
- Reason about when specialized or fine-tuned agents are justified versus when better control structure is the more practical intervention.

Starter Files (Included in Overleaf)

This Overleaf project includes the necessary script templates and environment checkouts in the `lab-9-files.zip` packet.

- `nanoagent.py`: A stand-alone coding assistant with tool support, adapted to target Purdue's GenAI Studio.
- `orchestrator.py`: An orchestration framework implementing three distinct agent roles (Planner, Implementer, Reviewer).

Author's Contact Information: Ricardo Andres Calvo Mendez, Purdue University, West Lafayette, Indiana, USA, rcalvome@purdue.edu.

Pre-Lab

2 Pre-Lab: Concepts, Reading, and Environment Setup

2.1 Goal

The goals of this pre-lab are to ensure that you:

(1) Understand and can articulate what agentic workflows, orchestration, and the CostToSuccess metric mean
(2) Understand the potential overhead and risks of utilizing multi-agent systems
(3) Can run the `nanoagent` and custom orchestrator locally using the Purdue GenAI Studio

2.2 Readings & Definitions

The following resources are not required, but provide useful context and vocabulary for the lab. If you are unfamiliar with multi-agent orchestration, we recommend skimming at least one before class.

- LangGraph Overview: High-level overview of the LangGraph framework.
- LangGraph Agentic Concepts: roles, cycles, and how agents share state
- LangGraph Multi-Agent Systems: how orchestration manages workflows across specialized agents
- LangGraph Persistence: How LangGraph agents persist context
- LangGraph Introductory Video: a short walkthrough of the above concepts

While we will *not* use the LangGraph as it is a heavier framework than is required for this lab, the reading provides a necessary foundation on how agents interact, cycle, and share state. The goal of this reading is to provide a background on the potential benefits and drawbacks of multi-agent orchestration, and to familiarize you with the terminology used in this lab.

2.2.1 Single-Agent vs. Orchestrated Workflow. In a **Single-Agent Workflow**, a user provides a single, unified prompt to a Large Language Model (LLM) which has access to a set of tools. The LLM iterates in a single continuous loop until it believes it has solved the problem. In contrast, an **Orchestrated Workflow** breaks the task into specialized, bounded roles. For example, a *Planner* creates a step-by-step approach, an *Implementer* writes code and runs tests strictly following the plan, and a *Reviewer* accepts or rejects the changes based on quality and completeness.

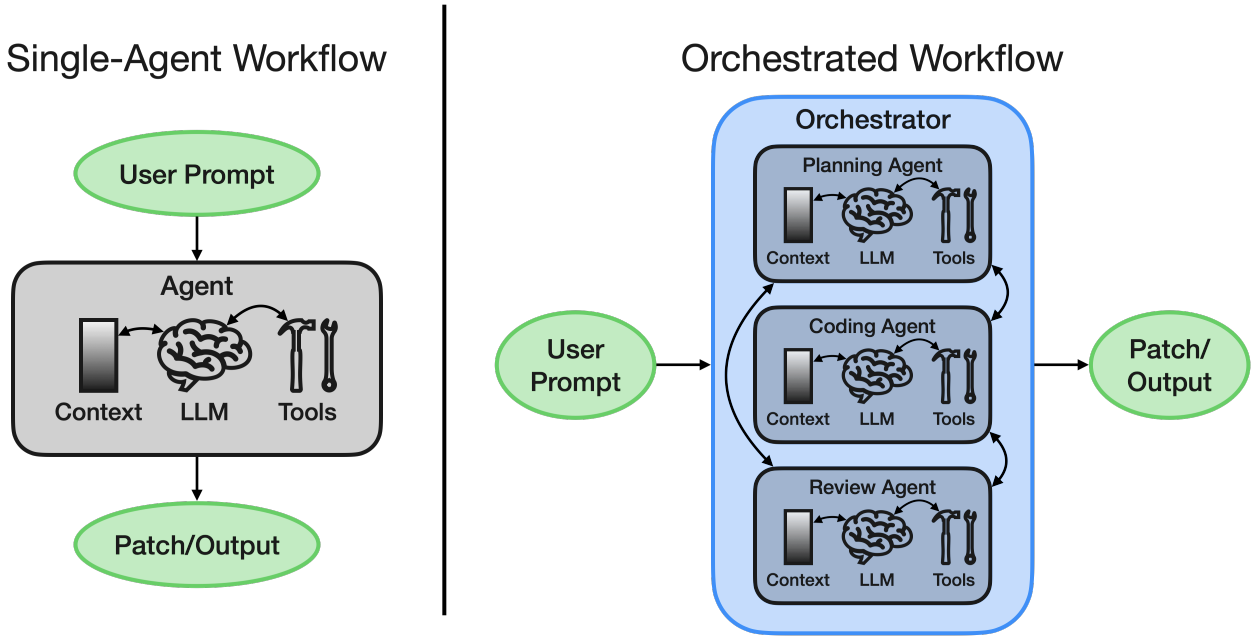

Fig. 1. Single-Agent vs. Multi-Agent Orchestrated Workflow

2.2.2 CostToSuccess. When evaluating LLMs for software engineering, "the output looked good and passed the tests" is an incomplete metric. To complement it, this lab measures **CostToSuccess**. It is defined as the total number of tokens (input + output) spent by the system until the *first acceptable solution* is produced. A method that does not produce a mathematically or programmatically acceptable solution within the strict step/budget cap automatically loses, even if the tokens spent up to that point were minimal.

2.2.3 Role Separation and Hard Bounds. Role separation limits the scope of what an individual LLM call has to reason about. However, this separation requires the system to pass context back and forth, consuming extra tokens (orchestration overhead). To prevent infinite loops and "token spiraling," an orchestrator relies on **Hard Bounds** (e.g., a maximum of 3 planner passes or 6 implementer passes). Hard Bounds are the strict, pre-defined numerical limits placed on an agentic system's execution to ensure it terminates and remains cost-efficient.

In professional practice, these bounds are not chosen arbitrarily. Engineers typically run the system on a suite of representative tasks, observe the point at which the model typically converges on a correct solution, and use the **CostToSuccess** metric to tune these limits. The goal is to find the "sweet spot" where additional passes no longer justify the marginal token expenditure.

2.3 Environment Setup

(1) **Linux Setup**
This lab requires a Linux environment to run the evaluation and checkout scripts properly. If you are on Windows, you **must** use Linux, Windows Subsystem for Linux (WSL), a Virtual Machine, or AWS Free Tier.

(2) **Option 1: You use Linux. Skip to Step 5.**

(3) **Option 2: Create a Local VM with VirtualBox** *The following steps are expected to be generally correct, but YMMV. Read some docs or chat with an LLM as needed.*

- **Install VirtualBox**
  - Go to VirtualBox and download normally
- **Download Ubuntu ISO**
  - Go to Ubuntu
  - Download the latest version
  - This will download an ISO file. Put it somewhere memorable, but **DO NOT OPEN IT**
- **Create the VM**
  - Open VirtualBox
  - Click **New** and fill in:
    - Name: `Ubuntu`
    - Folder: location where the VM will be stored
    - ISO image: select the file downloaded earlier
  - Add a username and password (MUST be changed from the defaults) that you will remember
  - Allocate 8 GB RAM and at least 4 CPUs
  - Click **Create a Virtual Hard Disk Now** and choose at least 25 GB
  - Click **Next** to begin installation. If the system does not automatically reboot after install, shut down the VM and start it again

(4) **Option 3: Create a Free Tier VM on AWS** *The following steps are expected to be generally correct, but YMMV. Read some docs or chat with an LLM as needed.*

- **Sign up / Log in to AWS**
  - Go to AWS and log in, or create a free account
- **Launch a New EC2 Instance**
  - Navigate to the EC2 Dashboard and click **Launch Instance**
- **Choose the OS**
  - Select **Amazon Linux** (Free tier eligible)
- **Select Instance Type**

  - Choose `t3.micro` (Free tier eligible)
- **Create Key Pair for SSH Access (Optional)**
  - Create a new key pair (choose `.pem` format)
  - Download the key and store it securely
- **Configure Security (Optional**
  - Allow SSH access (port 22) from your IP address
- **Launch the Instance**
  - Click **Launch Instance** to start your VM
- **Connect to the VM**
  - You can connect to the VM through a browser on the AWS website (Instances > Connect), or if you've set up SSH Access, you can use your terminal or command prompt:
    `ssh -i <key>.pem <user>@<public-ip>`
  - Replace `<user>` with `ubuntu` for Ubuntu or `ec2-user` for Amazon Linux.
  - Replace `<key>` with your downloaded .pem file and `<public-ip>` with the instance's public IP

(5) **Download Starter Files**

Download and unzip `lab-9-files.zip`, attached to this Overleaf project.

(6) **Obtain a Purdue GenAI Studio API Token**

(a) Go to the Purdue GenAI Studio page and sign in with your Purdue SSO Credentials.

(b) Navigate to **Profile** > **Settings** > **Account**.

(c) In the API keys section, generate a new API Key or copy one of your existing ones. (`<your_token>`).

(7) **Set up a Python Virtual Environment**

```
(1) cd /path/to/your/lab-9-files
# With python + pip
(2) python3 -m venv llm-agent-env
(3) source llm-agent-env/bin/activate # (Windows) llm-agent-env\Scripts\activate
(4) pip install requests python-dotenv
# With uv
(2) uv sync
```

(8) **Configure your Environment Variables**

```
(1) touch .env
(2) echo "API_KEY=your_key_here" >> .env
(3) echo "API_URL=https://genai.rcac.purdue.edu/api/chat/completions" >> .env
(4) echo "MODEL=gpt-oss:120b" >> .env
```

2.4 Python Setup (Student Starter Files)

You are given a starter bundle (`lab-9-files.zip`) containing `nanoagent.py` and `orchestrator.py`. Unzip the files and complete the sanity check to ensure your environment is correctly configured to run the lab.

2.4.1 Sanity Check. Run the following commands to ensure your GenAI Studio token is working and that `nanoagent` successfully logs token usage.

```
(1) python3 nanoagent.py --help
(2) python3 nanoagent.py --log test_run.jsonl # Input your prompts
(3) "Ctrl + C" to exit
(4) cat test_run.jsonl
```

You should see a JSON output containing the task ID, the role, and the token counts:

{"task_id": "interactive", "condition": "single", "role": "single", "input_tokens": 499, "output_tokens": 74, "total_tokens": 573, "cumulative_total": 573, "stop_reason": "tool_calls"}

2.5 Pre-Lab Tasks

This lab is intended to explore the engineering trade-offs of multi-agent systems. Open `nanoagent.py` and familiarize yourself with the adapter layer (`run_tool()`, `make_studio_tools()`). Note how the `call_api()` function captures `input_tokens`, `output_tokens`, and `total_tokens` and writes them to a local JSONL trace file.

Deliverable 1 (Code Analysis)

(1) **Tool Execution:** In `nanoagent.py`, what is the specific responsibility of the `run_tool()` function? How does it bridge the gap between the LLM's JSON response and the local file system?

(2) **The Agent Loop:** Describe the exit condition for the main loop in `nanoagent.py`. Under what circumstances does the agent stop calling the model?

(3) **Context Handoffs:** In `orchestrator.py`, how is context passed between the **Planner**, **Implementer**, and **Reviewer**? Does the Reviewer see the entire message history of the Implementer, or a filtered summary?

(4) **Token Tracking:** Locate the usage logging in `call_api()`. How are the cumulative totals for a task calculated when multiple agent roles are involved in a single workflow?

Write **8–12 sentences total**. Keep it concise, analytical, and technical.

INSERT ANSWER HERE

Answer the following in the space below based on the assigned readings and the lab context.

Deliverable 2 (Concept Review)

(1) What specific problem is orchestration trying to solve that a single long prompt does not?

(2) What are the potential downsides of adding multiple specialized roles to an LLM workflow?

(3) What would you consider to be concrete, measurable evidence that orchestration is worth its overhead?

Write **6–8 sentences total**. Keep it concise, analytical, and technical.

INSERT ANSWER HERE

In-Class Lab

1 In-Class Lab: Evaluating Orchestration vs. Single-Agent Baselines

1.1 Overview

In this in-class lab, you will measure the marginal value of imposed control by comparing a single-agent baseline against an orchestrated multi-agent workflow.

You will execute both conditions across three issues of increasing complexity. For each attempt, your workflow will interact with the local repository and generate a log of its API calls. By recording the input and output tokens spent until an acceptable solution is produced, you will track the **CostToSuccess**.

The in-class component is strictly timeboxed. The goal is to collect data on agent behavior under constraints, **not to successfully fix every bug**. If your timebox expires before the agent resolves the issue, stop the run, record the failure, and move on.

Table 1. Overview of the textstats GitHub repository and issues

Task	Description / Summary	Source Repository	Issue / PR Link
Task A (Simple)	Consecutive spaces incorrectly inflate the total word count	textstats (task-simple branch)	Simple Issue
Task B (Medium)	Aggressive punctuation normalization merges words and breaks contractions	textstats (task-medium branch)	Medium Issue
Task C (Hard)	Inaccurate word length calculations causing rounding errors and crashes	textstats (task-hard branch)	Hard Issue

The table above describes the issues that you will be tackling and introduce the textstats GitHub repository which contains the bugs. The textstats repository is a lightweight, self-contained Python library designed to compute basic text metrics using only Python's standard library. It features a core module with straightforward functions to analyze text—such as word and sentence counts, reading time estimates, and lexical diversity—alongside a corresponding pytest suite that verifies these behaviors.

1.2 Provided Files

- `nanoagent.py`: Your baseline single-agent LLM wrapper.
- `orchestrator.py`: Your multi-agent runner (Planner → Implementer → Reviewer).
- `tasks/`: A folder containing the three in-class tasks:
  - `task-simple/`: A small patch touching a single file.
  - `task-medium/`: A moderate patch requiring reasoning across multiple files.
  - `task-hard/`: A multi-part fix requiring deeper reasoning across multiple files.
- Inside each task folder, `checkout.sh` is provided to seed the repository state. Running `task-X/checkout.sh` will produce `task-X/repo`, which will be a clone of the `textstats` repository at a commit where there is a bug in the code.
  - `task-X/repo/run_tests.sh`: Run the tests (should fail initially).
  - `task-X/repo/ISSUE.md`: The issue description provided to the LLMs.

1.2.1 Workflow Structure. For each task, you will navigate to the task directory, run the checkout script, verify the bug exists using `run_tests.sh`, and then invoke the agent. The agent will interact with the codebase and output its token usage to a local JSONL trace. You will repeat this for both the single-agent (`nanoagent.py`) and the orchestrated (`orchestrator.py`) conditions.

1.3 Task 1: The Simple Fix (Task A)

1.3.1 Execution. Navigate to the `task-simple` directory. Run the following commands to test the baseline condition:[1]

For this lab, "acceptance" is defined as passing the provided test suite. This is a useful but limited acceptance criterion: passing the tests is necessary but it is not the same as establishing full correctness.

```
(1) cd tasks/task-simple/
(2) ./checkout.sh
(3) cd repo/ # Go into the task-simple branch that was cloned from textstats
(4) ./run_tests.sh  # Confirm the test fails
(5) cd ../../../ # Get back into the root directory
(6) python3 nanoagent.py --task ./tasks/task-simple/repo --condition single --log ./single.jsonl
(7) cat ./tasks/task-simple/repo/single.jsonl
```

Troubleshooting: If the run_test.sh fails to run due to pytest not being found replace the "pytest" with "python -m pytest". Timebox: 5-8 minutes. Stop the agent if it exceeds this time.

This is a good time to manually inspect the patch made by the agent. It should be very simple for the first task. Did the patch work (i.e. do the tests pass)? Note the total token usage, as well as any irregularities in the patch for the sake of comparing to the orchestrator. Next, reset the environment and test the orchestrated condition:

```
(1) cd tasks/task-simple/
(2) ./checkout.sh && cd ../../ # Reset the repo for orchestrated run
(3) python3 nanoagent.py --task tasks/task-simple/repo --condition orchestrated --log orchestrated.jsonl
(4) cat ./tasks/task-simple/repo/orchestrated.jsonl
```

Deliverable 1 (Simple Fix Record)

- **Single-Agent Baseline:** Was it accepted? Total Tokens? Number of Tool Calls?
- **Orchestrated Workflow:** Was it accepted? Total Tokens? Number of Tool Calls?
- **Compare the runs:**
  - Did the orchestrator introduce unnecessary overhead for this simple task? Use the cumulative token output from the logs.
  - Did either workflow produce/fail to produce/produce a partial solution (i.e. passes 2/3 tests)?
  - Did either workflow produce a decidedly higher quality patch?

Provide your token counts and answer the questions in **2-4 sentences**.

[1] In the interest of time, we only run each scenario once in the in-lab part. In the take-home we perform multiple runs to explore variability.

INSERT ANSWER HERE

1.4 Task 2: The Medium Fix (Task B)

1.4.1 Execution. Navigate to the `task-medium` directory. Repeat the process for the medium complexity task. This issue involves cross-referencing at least two files.

```
(1) cd tasks/task-medium/
(2) ./checkout.sh
(3) cd repo/
(4) ./run_tests.sh
(5) cd ../../../
(6) python3 nanoagent.py --task tasks/task-medium/repo --condition single --log single.jsonl
(7) cat tasks/task-medium/repo/single.jsonl
```

Timebox: 8-12 minutes. Stop the agent if it exceeds this time. Next, reset the environment and test the orchestrated condition:

```
(1) cd tasks/task-medium/
(2) ./checkout.sh && cd ../../
(3) python3 nanoagent.py --task tasks/task-medium/repo --condition orchestrated --log orchestrated.jsonl
(4) cat tasks/task-medium/orchestrated.jsonl
```

Deliverable 2 (Medium Fix Record)

- **Single-Agent Baseline:** Was it accepted? Total Tokens? Number of Tool Calls?
- **Orchestrated Workflow:** Was it accepted? Total Tokens? Number of Tool Calls?
- How did the search/reasoning behavior differ between the two conditions as the codebase footprint increased?

Provide your token counts and answer the questions in **2-4 sentences**.

INSERT ANSWER HERE

1.5 Task 3: The Hard Fix

1.5.1 Execution. Navigate to the `task-hard` directory. This is the hardest in-class task, requiring multiple steps and deeper reasoning.

```
(1) cd tasks/task-hard/
(2) ./checkout.sh
(3) cd repo/
(4) ./run_tests.sh
(5) cd ../../../
(6) python3 nanoagent.py --task ./tasks/task-hard/repo --condition single --log single.jsonl
(7) cat tasks/task-hard/repo/single.jsonl
```

Timebox: 10-15 minutes. Stop the agent if it exceeds this time. Next, reset the environment and test the orchestrated condition:

```
(1) cd ./tasks/task-hard/
(2) ./checkout.sh && cd ../../
(3) python3 nanoagent.py --task tasks/task-hard/repo --condition orchestrated --log orchestrated.jsonl
(4) cat tasks/task-hard/orchestrated.jsonl
```

Deliverable 3 (Hard Fix Record)

- **Single-Agent Baseline:** Was it accepted? Total Tokens? Number of Tool Calls?
- **Orchestrated Workflow:** Was it accepted? Total Tokens? Number of Tool Calls?
- Did the single-agent baseline "wander" or get stuck in a loop without the Planner/Reviewer roles to guide it?

Provide your token counts and answer the question in **2-4 sentences**.

INSERT ANSWER HERE

1.6 Task 4: Tabulate Results

We will want to tabulate our results from running both agent workflows to report our results. Use the table below to store the information from each run. Be sure to note expected and unexpected behavior.

Deliverable 4 (Tabulate Results)

Task	Condition	Accepted?	Total Tokens	Model Calls	Notes
Simple	Single-agent				
	Orchestrated				
Medium	Single-agent				
	Orchestrated				
Hard	Single-agent				
	Orchestrated				

NOTE: *All of the fields that are needed for the table can be found in the .jsonl file that was generated for each run.*

1.7 Task 5: In-Class Synthesis Reflection

Now that you have run all six conditions, aggregate your findings to evaluate our core hypothesis regarding CostToSuccess.

Deliverable 5 (Synthesis Reflection)

- For which task, if any, did orchestration function primarily as overhead rather than as useful control?
- For which task, if any, did orchestration begin to justify its added cost?
- Where did the single-agent baseline waste the most effort? Where did the orchestrated workflow waste the most effort?
- Did this exercise affect your judgment about when a single-agent workflow is sufficient and when orchestration is warranted? Explain briefly.

Answer questions in **4-8 sentences** based on the data you collected.

INSERT ANSWER HERE

Take-Home Lab

1 Take-Home: Orchestration at Scale and Reflection

1.1 Goal

In the In-Class Lab, you executed a timeboxed evaluation of simple to hard tasks to observe the overhead of multi-agent systems. In the Take-Home Lab, you will implement a deeper dive. Your goal is to pinpoint the specific complexity crossover point where the single-agent baseline's tendency to "wander" makes it more expensive (in **CostToSuccess**) than the heavy token overhead of the orchestrated workflow.

1.2 Background

1.2.1 The Search Space Problem. As a software task grows in scope (*e.g.* spanning multiple files or requiring specific sequencing), the search space for a valid patch expands exponentially. A single-agent baseline attempts to hold the entire context, the plan, and the execution in a single prompt loop. As the reasoning burden increases, single agents are prone to hallucinating non-existent functions, modifying the wrong files, or getting stuck in repetitive error loops.

1.2.2 Decomposition and Bounding. Orchestration combats the search space problem through *decomposition* and *bounding*.

- **Decomposition:** The Planner agent breaks the large issue into a step-by-step checklist.
- **Bounding:** The Implementer agent is constrained. It only sees the current step and is restricted to a small step budget. If it fails, the Reviewer intervenes rather than letting the Implementer spiral out of control.

1.2.3 The Crossover Hypothesis. Because the Planner and Reviewer consume tokens just to "talk" to the Implementer, orchestration has a high fixed cost. Our core hypothesis is that for a small bug, this fixed cost wastes money. But for a complex bug, the cost of the single-agent making dozens of failed, unbounded attempts will eventually exceed the fixed cost of orchestration.

1.3 Provided Files

For this portion of the lab, you will use two new task environments provided in your starter zip.

- `tasks/task-takehome-moderate/`: A bug that spans one main file plus one secondary file. The issue description is slightly underspecified, forcing the agent to inspect the code before proposing a fix.
- `tasks/task-takehome-complex/`: An issue requiring explicit decomposition across multiple subtasks (*e.g.* modifying parsing logic *and* updating the corresponding CLI help text).

1.4 Knobs and Levers

In this section of the lab, you are encouraged to examine the constants defined within the `TaskConfig` struct on line 46 of `task_support.py`. Modifying the following values will affect the token usage and peak capabilities of both the single and the orchestrated agent:

- `max_implementer_passes`: How many attempts does the implementer agent get? (default=3)
- `max_implementer_steps`: How many model calls can the implementer make each attempt? (default=15)
- `max_total_tokens`: What is the agent's total token budget (default=75,000)
- `max_single_agent_turns`: How many iterations can the single agent perform? (default=8)
- `max_tool_iterations`: How many tool calls can either a single agent or an implementer make (default=16)

There are more variables in the `TaskConfig` struct, but these are the most proximally related to the agents' token expenditure and capabilities. In general, increasing any of these values will increase the number of tokens spent per task but will allow the program to complete more complex tasks in a single run.

1.5 Task 1: Moderate Complexity

In this task, the bug spans 1-2 files and requires localized reasoning. Simple pattern-matching will likely fail. You must run both your `nanoagent.py` baseline and your `orchestrator.py` workflow until they either pass the acceptance tests or exhaust their hard

step limits. In order to reduce non-determinism within the results obtained, you are to run both nanoagent.py and orchestrator.py individually 3 times. After completing all 6 runs reflect on the results in Deliverable A.

Deliverable A (Moderate Complexity)

- **Single-Agent Baseline:** Was the proposed solution accepted for all the runs or just some? What were the Average Total Tokens counts for each run? Average Number of Tool Calls for each run?
- **Orchestrated Workflow:** Was the proposed solution accepted for all the runs or just some? What were the Average Total Tokens counts for each run? Average Number of Tool Calls for each run?
- **Reflection:** Did orchestration start to pay for itself here, or is a 1-2 file fix still too trivial to justify the overhead? Briefly explain the behavior you observed across the runs.

Provide your token counts and answer the questions in **3-4 sentences per question**.

INSERT ANSWER HERE

1.6 Task 2: Higher Complexity (Decomposition)

This task requires changes across 2-3 interacting files. There is at least one point where planning matters (*e.g.* if the agent updates the parser but forgets the CLI text, the acceptance test will fail). Again in order to reduce non-determinism within the results obtained, you are to run both nanoagent.py and orchestrator.py individually 3 times. After completing all 6 runs reflect on the results in Deliverable B.

Deliverable B (Higher Complexity)

- **Single-Agent Baseline:** Was the proposed solution accepted for all the runs or just some? What were the Average Total Tokens counts for each run? Average Number of Tool Calls for each run?
- **Orchestrated Workflow:** Was the proposed solution accepted for all the runs or just some? What were the Average Total Tokens counts for each run? Average Number of Tool Calls for each run?
- **Analyze the crossover:** Was the orchestrated workflow finally cheaper in CostToSuccess than the baseline? How did the Planner's output prevent the Implementer from wandering?

Provide your token counts and answer the questions in **3-4 sentences per question**.

INSERT ANSWER HERE

1.7 Task 3: Comprehensive Reflection

Based on all the data you have collected from the In-Class and Take-Home portions, answer the following reflection questions.

1.8 Part 1: Limits of LLMs

Deliverable C (Limits of LLMs)

(1) In your experiments, what were the dominant failure modes?
(2) Did failures tend to be obvious syntax errors, or plausible-looking logical errors?
(3) How stable were outcomes across reruns? What does this suggest about the limits of evaluating LLMs by a single benchmark run?

Answer the questions in **2-3 sentences per question**.

INSERT ANSWER HERE

1.9 Part 2: Value of Control

Deliverable D (Value of Control)

(1) What concrete benefit, if any, did role separation provide?
(2) Did the Planner/Reviewer roles actually improve correctness, or did they mostly just improve the polish/formatting of the output?
(3) What specific kinds of software engineering tasks seem to benefit most from bounded decomposition and review?

Answer the questions in **2-3 sentences per question**.

INSERT ANSWER HERE

1.10 Part 3: Cost and Engineering Tradeoffs

Deliverable E (Cost and Engineering Tradeoffs)

(1) Overall, for which task was the single-agent baseline the definitive winner on CostToSuccess? For which task was orchestration the winner?
(2) If orchestration lost on a complex task, why? Was it due to too much coordination chatter, bad decomposition, redundant reviews, or something else?
(3) How sensitive are your conclusions to the exact acceptance tests provided? If the tests were weaker, would the single-agent have "succeeded" faster?

Answer the questions in **2-3 sentences per question**.

INSERT ANSWER HERE

1.11 Part 4: Specialized / Fine-Tuned Agents

Deliverable F (Specialized vs Fine-Tuned Agents)

(1) Under what circumstances would prompt-specialized agents (*e.g.* an agent strictly prompted to only write SQL queries) seem justified?
(2) Under what circumstances would training a fine-tuned model seem justified rather than overkill? Consider properties like task volume, stable I/O formats, and the cost of failures.
(3) For the tasks in this lab, does fine-tuning seem worthwhile, or is a better control structure (orchestration) the more practical and cost-effective intervention?

Answer the questions in **2-3 sentences per question**.

INSERT ANSWER HERE

References